포트폴리오

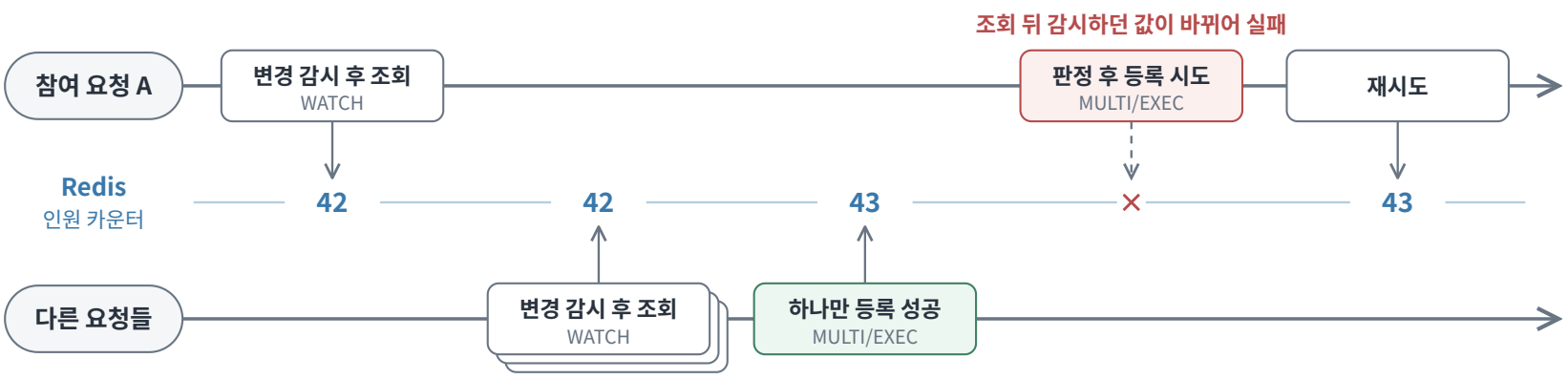
커머스 선착순 구매와 마케팅 파이프라인

선착순 이벤트 트래픽을 처리하고, 페이지 조회, 스크롤 깊이, 체류 시간, 구매 같은 행동 데이터를 수집해 마케팅 대시보드와 쿠폰, 알림 발송에 활용하는 프로젝트입니다.

- GitHub: https://github.com/NileTheKing/marketing-intelligence-platform

1. 선착순구매API가 동시요청시 낙관적락 충돌 재시도로 응답이 지연되는 문제를 Lua script을 이용하여 응답시간 개선

전체적인 구조



요청이 많을수록 매번 하나만 성공하고 나머지는 재시도

문제 원인

- 한정 수량 상품의 선착순 판매에서, 중복 참여와 정원 초과를 막으면서 오픈 순간 몰리는 요청에 빠르게 참여 결과를 반환해야 함
- Redis의 참여자 Set과 인원 카운터로 중복 여부와 정원을 판정하고, 낙관적 락(WATCH + MULTI/EXEC)으로 참여자 등록과 인원 변경을 원자적으로 처리하던 구조
- 목표 부하 요구사항이 높아지면서 낙관적 락은 같은 카운터를 변경하는 요청 간 충돌과 재시도가 증가해, 선착순 참여 API 응답이 지연되는 문제 발생

해결 과정

중복 참여와 정원 초과를 제어하면서 충돌 재시도가 없는 처리 방식 비교

SADD와 INCR 개별 호출

- SADD로 중복 참여를 확인하고, INCR로 증가한 인원 수를 정원과 비교. 초과하면 참여자 등록과 인원 증가를 되돌려 동시성 제어
- 명령 사이 중단으로 상태가 일부만 반영될 수 있어, 불일치 감지와 사후 복구 설계 필요
- 여러 번의 Redis 호출로 참여 API 응답 p95도 Lua보다 약 32% 길어, 복구 부담과 응답 성능을 고려해 제외

Lua 선택

- 참여자 등록, 인원 증가, 정원 초과 시 되돌리기를 원자적으로 실행해 명령 사이의 상태 불일치를 방지하고, 낙관적 락의 충돌 재시도도 제거
- 한 번의 Redis 호출로 판정을 완료해 개별 호출 대비 참여 API 응답 p95 24.5% 단축. 원자성과 응답 성능을 함께 확보할 수 있어 선택

테스트

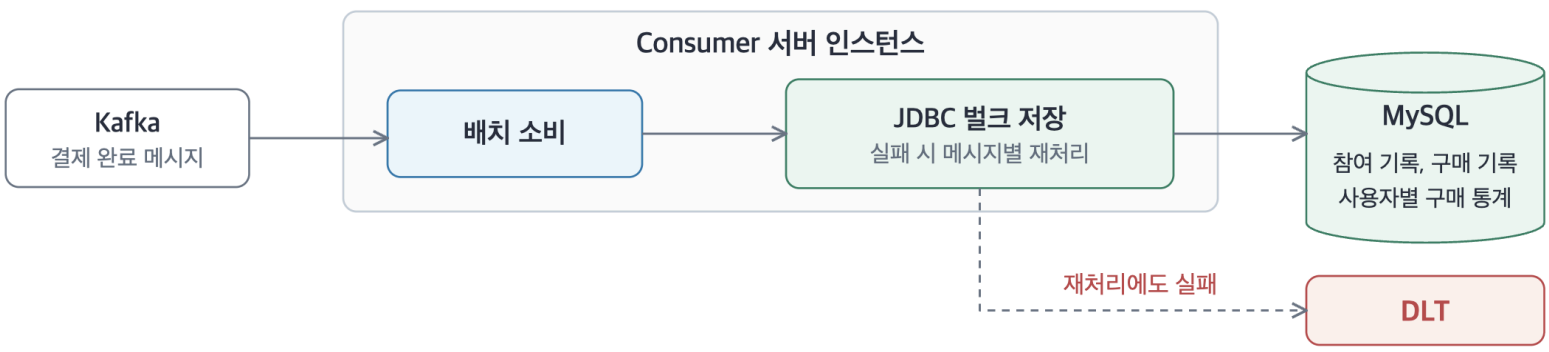
- 성능 비교**:** 로컬 Docker 환경에서 요청 300건, 정원 100명, 동시 호출 64개로 예열 후 방식별 3회 측정. 선착순 참여 API 응답 비교
- 정합성 검증**:** 동일 사용자 재요청과 정원 초과 요청의 판정 확인. 명령 사이 두 지점에서 각각 50건을 중단한 뒤 재요청해 참여자 수와 카운터 일치 확인

결과

- WATCH 대비 참여 API 응답 p95 84.8% 단축해 낙관적 락의 충돌 재시도 비용 제거
- 변경한 구조를 배포한 클라우드 VM에서 3,000명 선착순 API 부하 테스트 시 peak 약 1,000 RPS, p95 500ms로 응답하고 정원 초과 판매 0건
- 중복 재요청, 정원 초과 요청, 명령 사이 중단 후 재요청 모두 참여자 수와 카운터 일치

2. 결제 후속 처리 Kafka Consumer의 처리 지연 문제를 Jdbc Bulk Insert를 이용하여 TPS 2.2배 및 처리량 해소

전체적인 구조



문제 원인

- 선착순 판매의 결제 완료 메시지를 Kafka로 받아 구매 기록과 집계 데이터를 비동기 반영. 피크에는 성공 메시지가 초당 약 300건 유입돼, 단건 처리 약 77 TPS로는 적체 해소에 한계
- JPA 배치를 적용해 약 430 TPS로 개선했지만, 피크 유입 대비 처리 여유가 크지 않아 집중된 메시지를 더 빠르게 해소할 필요
- 배치로 처리하더라도 INSERT와 UPDATE는 건별 실행돼, 메시지마다 SQL 실행 비용이 반복되던 구조

해결 과정

1. 기존 Consumer 서버 인스턴스 내 병렬 소비 확대를 시도

- VM의 가용 자원이 부족해 Consumer 서버 인스턴스를 추가하기 어려워, 기존 서버 인스턴스 안에서 소비 스레드를 늘려 Kafka 파티션을 병렬 처리하는 방향 검토
- 3개 파티션을 동시에 소비하도록 스레드를 1개에서 3개로 늘렸지만, 처리량 증가는 약 8%에 그치고 CPU 사용량도 할당량에 근접
- JFR에서 반복적인 SQL 준비와 Hibernate 실행 비용을 확인해, 병렬성 확대보다 메시지당 저장 비용을 줄이는 방향으로 전환

2. JPA 배치와 JDBC 벌크 저장 비교

JPA 배치 유지

- 조회와 트랜잭션을 묶어 단건 약 77 TPS에서 약 430 TPS로 개선
- INSERT와 UPDATE는 건별 실행돼 반복 SQL 비용이 남고, 피크 유입 대비 처리 여유를 더 확보할 필요

JDBC 벌크 선택

- 참여 기록과 구매 기록을 다중 행 INSERT하고, 구매 집계값을 일괄 UPDATE해 SQL 실행 횟수 감소
- 새로 생성된 PK를 후속 처리에 전달할 필요가 없어 다중 행 INSERT 적용 가능
- SQL을 직접 관리하는 비용을 감수하되, JPA 배치 대비 처리량이 약 2.2배 높아 JDBC 벌크 선택

3. 배치 확대에 따른 실패 범위 관리

- 묶어서 저장할수록 한 건의 오류가 같은 배치의 정상 메시지까지 롤백시키는 범위도 커짐
- 배치 실패 시 메시지 단위로 다시 처리해 정상 메시지는 저장하고, 끝까지 실패한 메시지만 DLT로 격리
- 이 재처리 비용을 포함해도 이득이 남는지 확인하기 위해 오류 메시지 0.1%를 섞어 비교. JPA 배치 약 397 TPS, JDBC 벌크 약 814 TPS로 벌크 방식 우위 유지

테스트

- 성능 조건**:** OCI VM, Consumer 서버 컨테이너 1개, CPU 1.2코어, 메모리 3GiB. 병렬성 비교 외에는 소비 스레드 1개, poll당 최대 20건으로 고정
- 처리량 측정**:** 예열 후 Kafka에 정상 메시지 15,000건을 미리 적재하고, 참여 기록과 구매 기록이 DB에 저장될 때까지 측정
- 실패 처리**:** 메시지 15,000건 중 0.1%를 영구 입력 오류로 구성해 정상 메시지 저장과 최종 실패 메시지의 DLT 격리 확인. 별도 MySQL 통합 테스트에서 구매 INSERT 실패를 주입해 참여 기록과 구매 기록의 동시 롤백 및 재처리 확인

결과

- 동일 자원의 단일 Consumer에서 JPA 배치 약 430 → JDBC 벌크 약 956 TPS로 처리량 약 2.2배 개선
- 오류 메시지 0.1% 조건에서도 정상 메시지를 모두 저장하고 최종 실패 메시지만 DLT 격리, 최종 consumer lag 0

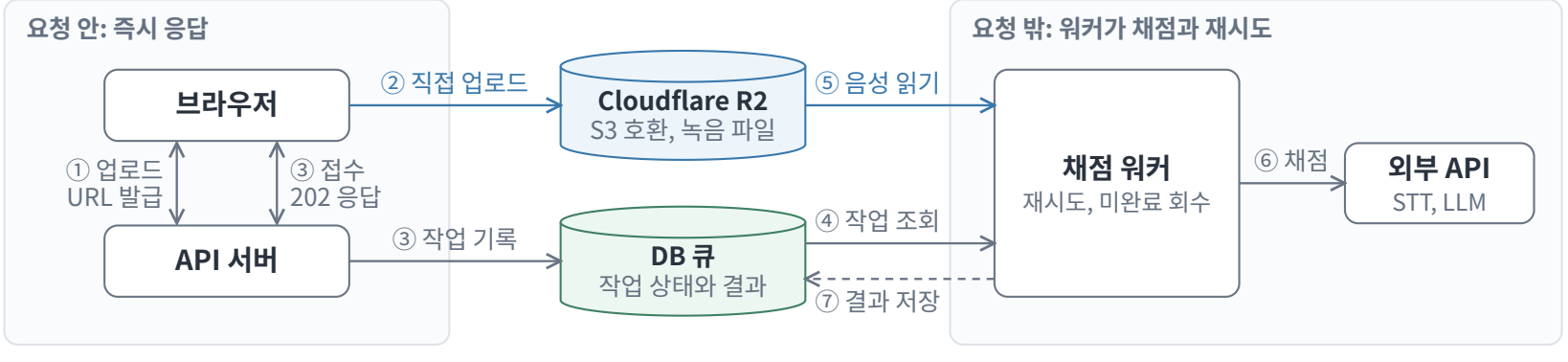
LLM 기반 OPIc 학습 플랫폼 OPIcnic

OPIc 말하기 시험 연습 서비스입니다. 녹음한 답변을 외부 STT와 LLM API호출로 채점해 피드백을 주고, 학습 기록을 모아 학습 방향을 제안합니다.

- GitHub: https://github.com/NileTheKing/opicnic

3. 파일처리를 외부API이용할시 I/O블로킹으로 인해 발생하는 힙메모리 부담을 S3 기반파일처리와 비동기처리로 개선

전체적인 구조



문제 원인

- OPIc 모의고사 15문항의 녹음 파일을 일괄 제출받아, 외부 음성 인식(STT) API와 LLM API로 답변을 변환하고 채점한 뒤 전체 결과를 동기 응답하는 구조
- 외부 API 일시 오류에 대응하기 위해 녹음 파일을 힙 메모리에 보관하며 재시도. 외부 호출이 지연되거나 재시도가 길어지면 응답 지연과 파일의 힙 점유가 함께 장기화되는 문제 발생
- 채점 상태도 메모리에서 관리해, 서버 재시작 시 미완료 작업 복구 불가

해결 과정

채점을 비동기 작업으로 분리해 접수 후 즉시 응답하고, 요청 종료와 서버 재시작 후에도 이어가도록 파일 보관 방식과 작업 큐 검토

1. 음성 파일 보관 방식
   - 선택 기준**:** 재시도할 음성을 보존하면서, 서버의 파일 수신과 힙 장기 점유를 줄일 수 있는가
   - 서버를 거쳐 저장하는 방식
     - 로컬 디스크, NFS, DB BLOB은 파일 보존이 가능하지만 애플리케이션 서버가 업로드를 받아 저장소로 전달해야 함
   - 클라이언트 직접 업로드 선택
     - S3 Presigned URL로 서버를 거치지 않고 업로드하면서 프로세스 밖에 음성 보존
     - S3 호환 스토리지 중 무료 용량과 이그레스 비용 조건에 맞는 R2 사용
   - 처리 중 메모리 관리
     - 워커가 처리할 음성만 읽고 재시도 대기 중에는 해제
2. 채점 작업 큐
   - 규모 산정: DAU 6만을 가정한 저녁 피크 제출량 약 6.7건/초. 제출당 평균 3.5문항으로 환산해, 초당 약 23문항의 처리 부하를 기준으로 큐 방식 검토
   - 대안 비교
     - 인메모리 큐: 서버 재시작 시 작업이 유실돼 제외
     - 별도 메시지 큐: 큐 운영과 메시지 처리 완료와 DB 저장 간 실패 처리 필요
     - DB 큐: 기존 DB에서 채점 결과 저장과 완료 상태 변경을 한 트랜잭션으로 처리할 수 있어 선택
   - 작업 복구
     - 완료한 문항까지 재처리하지 않도록 문항별 결과와 상태 저장
     - 재기동 시 미완료 문항을 회수하고, 접수 직후와 채점 중 강제 종료로 복구 확인

테스트

- 접수 응답: 클라우드 VM 배포 서버에서 동시 사용자 30명, 외부 API는 문항당 약 7.5초 걸리는 모의 서버로 대체. 파일 업로드와 채점 완료 시간을 뺀 서버 측 측정
- 힙 비교: 로컬 Docker 환경에서 동시 사용자 30명, 외부 호출 45% 실패 주입. 힙 512MB와 256MB 설정으로 비교
- 작업 복구: 접수 직후와 채점 중 서버를 강제 종료한 뒤 재기동

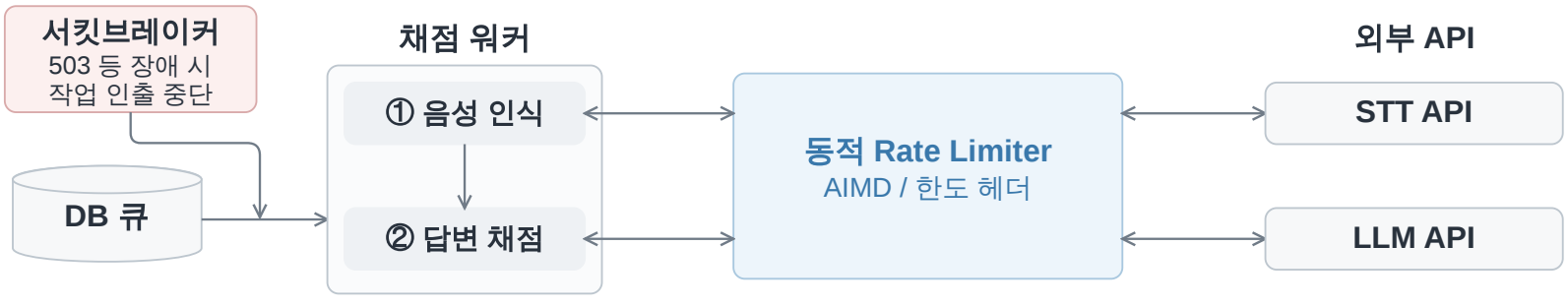
결과

- 동시 사용자 30명의 채점 요청에서 접수 응답 평균 18ms로, 외부 API 처리 완료를 기다리지 않고 응답
- 외부 API 장애 주입 부하에서 GC 후 최대 힙 사용량 375MB → 215MB로 43% 감소. 힙 256MB의 별도 실행에서는 OOM 18회 → 0회
- 접수 직후와 채점 중 서버 강제 종료 후 재기동 시 모의고사 15문항 채점 완료 확인

---

4. 외부 API 과부하 시 백로그 적체를 동적 Rate Limiter와 CircuitBreaker를 이용하여 백로그 해소 시간 50% 단축

전체적인 구조



문제 원인

- 외부 STT와 LLM API로 처리하는 채점 요청이 공채 시즌 피크에는 초당 약 46문항 유입되는 상황. 외부 제공자가 일시적으로 분당 호출 한도를 낮추면 유입량을 처리하지 못하고 채점 적체 발생
- 외부 API의 호출 한도가 처리량의 상한이어서, 워커 동시 처리 수를 늘려도 429와 재시도만 증가
- 기존에는 503 등 외부 API 장애 시 호출을 차단하는 서킷브레이커와, 복구 시 재시도를 분산하는 지수 백오프(full jitter)를 적용한 구조
- 호출 한도 감소로 발생한 429도 서킷의 실패율에 포함돼, 제공자가 처리할 수 있는 요청까지 차단하면서 채점 적체가 길어지는 문제 확인

해결 과정

1. 429를 서킷의 실패율 집계에서 제외했으나 재시도 폭풍 발생

- 남은 처리 용량을 활용하기 위해, 429를 서킷의 실패율 집계에서 제외한 설정으로 실험
- 429로 거절된 호출도 분당 호출 한도에 포함하는 제공자에서는, 반복 재시도가 채점을 완료하지 못하면서 정상 처리에 쓸 한도까지 소모
- 재시도 호출량이 복구된 제공자의 한도를 계속 초과해, 제공자 용량이 회복돼도 채점이 진행되지 않고 백로그 증가
- 지수 백오프(full jitter)는 재시도 시점을 분산하지만 전체 호출 속도를 제한하지는 못하므로, 제공자가 수용할 수 있는 속도로 호출량을 조절하는 방향 선택

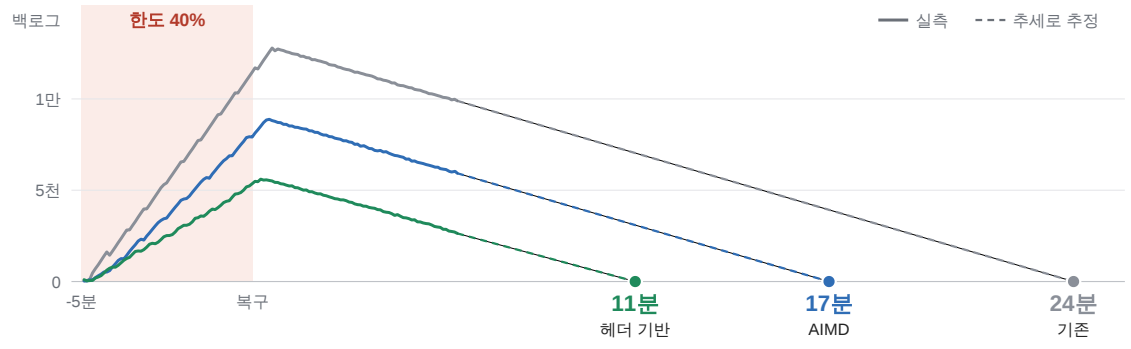
2. 제공자별 한도 정보에 맞춰 동적 Rate Limiter 적용

- 모델과 제공자 교체가 잦은 초기 단계라, 분당 호출 한도 정보를 제공하지 않는 경우에도 속도 제어가 필요
- AIMD 적용: 429 발생 시 호출 속도를 절반으로 낮추고 정상 응답이 이어지면 점진적으로 증가. 한도 정보 없이 응답을 기반으로 조절해, 기존 대비 백로그 해소 시간 약 30% 단축
- 헤더 기반 제어: 한도 정보를 제공하면 남은 요청 수와 초기화 시간에 맞춰 직접 조절. AIMD처럼 적정 속도를 탐색할 필요가 없어, 기존 대비 해소 시간 약 53% 단축
- 호출 속도는 Rate Limiter로 제어하고, 429는 서킷의 실패율 집계에서 제외해 과부하 제어와 장애 차단의 역할 분리

테스트

- 클라우드 VM에서 공채 시즌 피크인 초당 약 46문항 유입, 워커 동시 처리 상한 500문항으로 비교
- 제공자가 과부하로 분당 호출 한도를 정상의 40%로 낮췄다가 5분 뒤 복구하는 상황 재현. 거절된 호출도 한도에 포함
- 제공자 복구 후에도 유입을 유지하며, 속도 제한 방식과 서킷 설정별 백로그 변화 비교

결과



- 백로그 해소 시간 약 24분 → AIMD 약 17분으로 30%, 헤더 기반 제어 약 11분으로 53% 단축 (기존 서킷과 지수 백오프(full jitter) 조합 대비, 백로그 감소 추세 기반 추정)
- 제공자 용량 복구 시점의 백로그 약 1.14만 → AIMD 약 7.9천, 헤더 기반 제어 약 5.3천 문항으로 감소